Evaluating Contextual Amino Acid Embeddings for TCR–Epitope Binding Affinity Prediction

https://github.com/edwin-huang/CSE559TCREpitopeProject

Vignesh Mohan vmohanav@asu.edu
Ali Altaraif aebrahi7@asu.edu
Edwin Huang ehuang19@asu.edu
Skand Parvatikar sparvati@asu.edu

Abstract

Predicting T cell receptor (TCR) recognition of antigenic epitopes is a fundamental challenge in computational immunology, with direct implications for vaccine design, immunotherapy, and immune repertoire analysis. A key difficulty lies in evaluating model generalization to unseen TCRs and epitopes, as naive random splits often lead to overly optimistic performance estimates. In this work, we analyze splitting data to test on unseen TCR and epitope sequences using 3 different embedding models: catELMo, BLOSUM62, and BERT-Base-TCR. Performance is assessed using five repeated runs under both TCR-split and epitope-split settings, reporting AUC, precision, recall, and F1 score. Our results demonstrate that the model can generalize better to unseen TCRs than unseen epitopes, with the resulting AUCs in the TCR split being 5% to 10% higher.

1 Introduction

T cell receptor (TCR) recognition of peptide epitopes presented by major histocompatibility complex (MHC) molecules is central to adaptive immune responses. Accurately predicting TCR–epitope binding remains a challenging problem due to the complexities of protein folding and binding interactions, the extreme diversity of TCR variable chains and epitope peptides, and the limited overlap between experimentally observed sequences. Computational models must therefore generalize effectively to novel TCRs and epitopes that were not observed during training.

Recent work has shown that improper data splitting strategies can substantially overestimate predictive performance by allowing overlap between training and testing sequences. To address this issue, the ATM-TCR paper[1] proposed two biologically meaningful evaluation protocols: *TCR split*, where all test TCRs are unseen during training, and *epitope split*, where all test epitopes are unseen. Curiously, the paper finds that multiple models perform much better in the TCR split than the epitope split.

In this paper, we investigate this discrepancy with various embedding models not tested in the ATM-TCR paper, and discuss why the models might perform differently in the two different data splits.

2 Experimental Setup

2.1 Dataset and Splitting Strategy

We use a dataset of 300,016 TCR–epitope pairs with a balanced 1:1 ratio of positive and negative samples. In the TCR split, the training and testing sets do not share TCR sequences, in the epitope split they do not share epitope sequences. For both the TCR split and the epitope split, the training data is split into 80%

training data, and 20% testing data. Metrics for AUC, Precision, Recall, and F1 score are averaged over five repeated runs for each split, with their mean and standard deviation reported.

2.2 Embedding Strategy

To see whether our results apply with multiple embedding strategies, we use three different embedding models: catELMo[2], BERT-Base-TCR[3], and BLOSUM62[4].

catELMo is a bidirectional language model trained on amino acid sequences. BERT-Base-TCR is a transformer based language model. It uses the architecture in the TCRBert paper[3] but is trained on the same dataset catELMo was trained on. BLOSUM62 is substitution matrix for amino acids, but here is repurposed as an embedding matrix.

For each model and split, we train a downstream model with the same architecture: The TCR and epitope embeddings are each projected onto a 2048-dimension hidden layer, then concatenated and projected to a 1024 dimension layer, and projected to the output layer. SiLU was used as the activation function, and we used Batch Normalization and 0.3 rate dropout. This is the same architecture of the downstream model used by catELMo [2].

Model	Embedding Size	# Parameters in downstream model
BLOSUM62	20	4.293M
BERT-Base-TCR	768	7.356M
catELMo	1024	8.450M

3 Results

3.1 Overall Performance

Table 1 summarizes the AUC, Precision, Recall, and F1 metrics for all evaluated models. The metrics in the TCR split are consistently higher than those in the epitope split, indicating superior generalization to unseen sequences. The standard deviations in the epitope split are also much higher.

Table 1: Performance comparison across models (mean $\pm$ std over 5 runs).

Model	TCR Split AUC (%)	Epitope Split AUC (%)
BLOSUM62	75.03 ± 0.13	65.09 ± 1.99
BERT-Base-TCR	80.49 ± 0.23	69.27 ± 3.14
catELMo	$\mathbf{90.87 \pm 0.17}$	$\mathbf{85.00 \pm 2.35}$
Model	TCR Split Precision (%)	Epitope Split Precision (%)
BLOSUM62	68.10 ± 0.16	61.24 ± 1.67
BERT-Base-TCR	72.68 ± 0.20	65.34 ± 2.47
catELMo	$\mathbf{82.04 \pm 0.32}$	$\mathbf{76.46 \pm 1.96}$
Model	TCR Split Recall (%)	Epitope Split Recall (%)
BLOSUM62	68.05 ± 0.12	61.01 ± 1.59
BERT-Base-TCR	72.48 ± 0.10	64.06 ± 2.27

catELMo	**81.99 ± 0.28**	**76.12 ± 2.16**
Model	TCR Split F1 (%)	Epitope Split F1 (%)
BLOSUM62	68.03 ± 0.11	60.80 ± 1.59
BERT-Base-TCR	72.41 ± 0.09	63.28 ± 2.33
catELMo	**81.98 ± 0.29**	**76.03 ± 2.23**

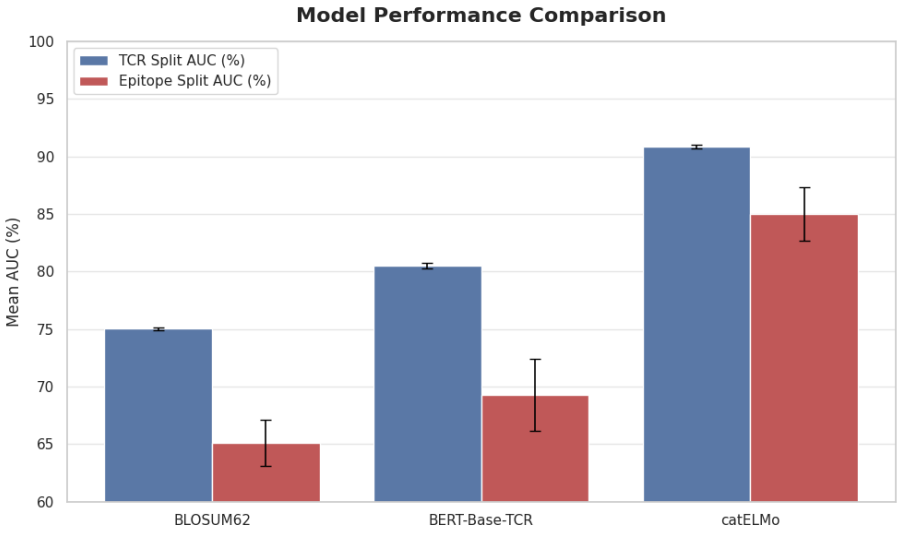


Figure 1: **Mean AUC performance of embedding strategies for each data splitting protocol.** Bars represent the mean Area Under the ROC Curve averaged over five independent runs for both TCR-split (blue) and Epitope-split (red) settings. Error bars denote the standard deviation.

4 Hyperparameter Choice

For consistency reasons, we kept most of the hyperparameters described in the catELMo paper [2]. We were not able to do a broad hyperparameter search because of resource constraints. The learning rate was set to 0.001, and we stop training if the validation loss does not improve for 30 epochs or after 200 epochs. The hidden layers had dimensions 2048 and 1024, respectively. We did significantly increase the batch size from 32 to 1024. Although this increased loss values, the final AUC scores did not decrease, which allowed us to train much faster.

5 Discussion

We notice that in all 3 embedding models, the AUC score in the TCR split is significantly higher than that in the epitope split. The standard deviation for the epitope split is also much higher. Running a program on the dataset, we observe that it contains 290,681 unique TCR sequences, but only 982 unique epitope sequences. Because there are many more TCR sequences, any TCR sequence will likely share patterns with others.

In contrast, the epitope split is harder because there are very little epitopes in the dataset. Many test epitopes may have patterns that are missing from the training epitopes, so the model cannot rely on familiar peptide features and performance drops. The model may also be capable of memorizing all the training epitopes without generalizing.

The ATM-TCR paper notes that a few epitopes make up a large portion of the dataset. Since the splitting algorithm chooses a fifth of the epitopes randomly, whether these frequent epitopes are in that fifth can drastically change the size of the test set. Of our 5 seeds, in the TCR split, the size of the test dataset varied from 59,783 to 60,241, close to the expected value of 60,004. However, the size of the test dataset for the epitope split varied from 27,761 (seed 46) to 101,593 (seed 45). We believe this is the primary cause of the high standard deviation in the epitope split. The AUC performance in the epitope split of all three embedding models were significantly worse in seed 45 which had 3 of the 4 most frequent epitopes in the test set, although they were not significantly better in seed 46.

Finally, the ranking of embedding models stays the same in both splits: catELMo performs best, followed by BERT-Base-TCR, and then BLOSUM62. However, all three models drop in performance in the epitope split, telling us that generalizing to unseen epitopes is the main challenge in this task.

6 Conclusion

This study demonstrates that models trained on a given TCR/Epitope dataset are able to generalize TCRs much better than epitopes. This may be a consequence of the dataset containing many more unique TCR strings than epitope strings, which is itself a consequence of the fact that TCR variable regions are more diverse than the epitopes found in the human body.

Because the dataset contains many more unique TCR strings than epitope strings, we suggest that the model is able to better resolve the underlying distribution of TCR sequences, while it is able to memorize the epitope sequences early in training. We also note that the epitope splitting method causes large variations in test datasets and metrics, caused by a heavy-tailed epitope frequency distribution.

References

[1] Michael Cai, Seojin Bang, Pengfei Zhang, and Heewook Lee. "ATM-TCR: TCR-Epitope Binding Affinity Prediction Using a Multi-Head Self-Attention Model". In: *Frontiers in Immunology* Volume 13 - 2022 (2022). ISSN: 1664-3224. DOI: 10.3389/fimmu.2022.893247. URL: https://www.frontiersin.org/journals/immunology/articles/10.3389/fimmu.2022.893247.

[2] Pengfei Zhang, Seojin Bang, Michael Cai, and Heewook Lee. "Context-Aware Amino Acid Embedding Advances Analysis of TCR-Epitope Interactions". In: (Apr. 2024). DOI: 10.7554/elife.88837.2. URL: http://dx.doi.org/10.7554/eLife.88837.2.

[3] Kevin E. Wu, Kathryn Yost, Bence Daniel, Julia Belk, Yu Xia, Takeshi Egawa, Ansuman Satpathy, Howard Chang, and James Zou. "TCR-BERT: learning the grammar of T-cell receptors for flexible antigen-binding analyses". In: *Proceedings of the 18th Machine Learning in Computational Biology meeting*. Ed. by David A. Knowles and Sara Mostafavi. Vol. 240. Proceedings of Machine Learning Research. PMLR, 30 Nov–01 Dec 2024, pp. 194–229. URL: https://proceedings.mlr.press/v240/wu24b.html.

[4] S Henikoff and J G Henikoff. "Amino acid substitution matrices from protein blocks." In: *Proceedings of the National Academy of Sciences* 89.22 (1992), pp. 10915–10919. DOI: 10.1073/pnas.89.22.10915. eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.89.22.10915. URL: https://www.pnas.org/doi/abs/10.1073/pnas.89.22.10915.